

✓ Homework 1 Part 2

This is an individual assignment.

Write your own code. You should repurpose any functions built during lecture.

```
1 # Import libraries and magics
2
3 import numpy as np
4 import matplotlib.pyplot as plt
5 %matplotlib inline
6 plt.style.use('bmh')
7 import pandas as pd
8
```

✓ Exercise 1 (15 points)

Consider synthetic noisy quadratic data of the form

$$t = x^2 + \epsilon$$

where ϵ is zero-mean additive Gaussian noise.

You will build a linear regression model that uses Laplacian basis functions as features:

$$\phi_j(x) = \exp\left\{-\frac{|x - \mu_j|}{b}\right\}$$

with centers $\mu \in \{-1.5, -0.5, 0.5, 1.5\}$ (for $j = 1, 2, 3, 4$), and a fixed scale parameter $b = 0.3$.

1. (8 points) **Generate a training set of $N_{\text{train}} = 60$ samples by drawing x uniformly from the interval $[-2, 2]$ and adding Gaussian noise with standard deviation 0.4. Fit a linear regression model using the Laplacian basis functions above. Include an intercept term. Complete the functions below while adhering to their format.** ✦

```

1 def NoisyQuadraticData(N, a, b, sigma):
2     '''Generates N data points in the range [a,b) sampled f
3     with additive zero-mean Gaussian random noise with standa
4
5     # N input samples, evenly spaced numbers between [a,b)
6     x = np.linspace(a, b, N, endpoint=False)
7
8     # draw N sampled from a univariate Gaussian distribution
9     noise = np.random.normal(0, sigma, N)
10
11     # desired values, noisy quadratic
12     t = x**2 + noise
13
14     return x, t

```

```

1 def LaplacianRegression(x,t):
2     '''Fit a regression model with Laplacian features to the
3
4     # Compute feature matrix X with Laplacian basis function
5     # Centers mu = [-1.5, -0.5, 0.5, 1.5] and scale paramete
6     mu = np.array([-1.5, -0.5, 0.5, 1.5])
7     b = 0.3
8
9     # Create feature matrix with intercept term and Laplacian
10    N = len(x)
11    X = np.ones((N, 5)) # First column for intercept
12
13    for j in range(4):
14        X[:, j+1] = np.exp(-np.abs(x - mu[j]) / b)
15
16    # Compute the solution for the parameters w using pseud
17    w = np.linalg.pinv(X) @ t
18
19    # Compute model prediction
20    y = X @ w
21
22    return w, y

```

```

1 # Generate training data
2 x_train, t_train = NoisyQuadraticData(60, -2, 2, 0.4)
3
4 # Fit the model
5 w, y_train = LaplacianRegression(x_train, t_train)
6
7 print("Model parameters:", w)

```

```
Model parameters: [ 1.74300136  1.15717391 -2.36929517 -2.44389347  1.25021377]
```

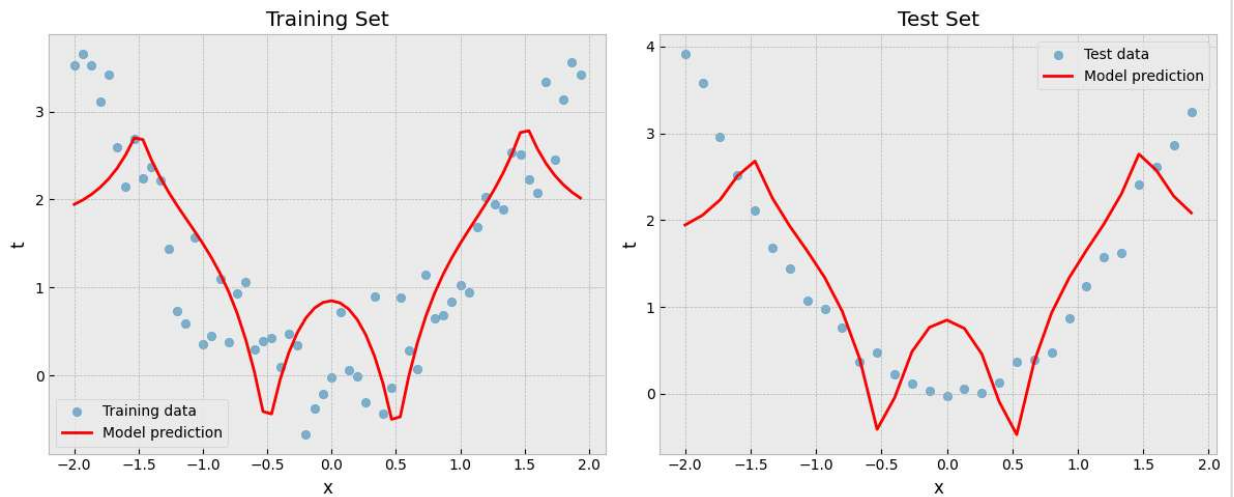
2. (7 points) **Generate an independent test set of $N_{\text{test}} = 30$ points from the same process with noise standard deviation 0.1. Use your fitted model to predict on the test set and report the test mean-squared error.**

```

1 # Generate test set (30 points, noise std = 0.1)
2 x_test, t_test = NoisyQuadraticData(30, -2, 2, 0.1)
3
4 # Use trained model to predict on test set
5 # Need to compute features for test data using same basis for
6 mu = np.array([-1.5, -0.5, 0.5, 1.5])
7 b = 0.3
8
9 # Create feature matrix for test data
10 N_test = len(x_test)
11 X_test = np.ones((N_test, 5)) # First column for intercept
12
13 for j in range(4):
14     X_test[:, j+1] = np.exp(-np.abs(x_test - mu[j]) / b)
15
16 # Make predictions using trained weights
17 y_test = X_test @ w
18
19 # Calculate test mean-squared error
20 test_mse = np.mean((y_test - t_test)**2)
21
22 print(f"Test Mean-Squared Error: {test_mse:.4f}")
23
24 # Optional: Plot results
25 plt.figure(figsize=(12, 5))
26
27 plt.subplot(1, 2, 1)
28 plt.scatter(x_train, t_train, alpha=0.6, label='Training data')
29 plt.plot(x_train, y_train, 'r-', label='Model prediction')
30 plt.xlabel('x')
31 plt.ylabel('t')
32 plt.title('Training Set')
33 plt.legend()
34
35 plt.subplot(1, 2, 2)
36 plt.scatter(x_test, t_test, alpha=0.6, label='Test data')
37 plt.plot(x_test, y_test, 'r-', label='Model prediction')
38 plt.xlabel('x')
39 plt.ylabel('t')
40 plt.title('Test Set')
41 plt.legend()
42
43 plt.tight_layout()
44 plt.show()

```

Test Mean-Squared Error: 0.4947



✓ Exercise 2 (15 points)

A city ride-share company wants to forecast the average number of trips per hour for the upcoming weekend. They model the hourly trip count as a Poisson random variable with unknown rate λ ($\lambda > 0$). The Poisson PMF is:

$$p(x|\lambda) = \frac{\lambda^x e^{-\lambda}}{x!}, \quad x \in \{0, 1, 2, 3, \dots\}$$

The analytics team places a Gamma prior on λ with parameters $\alpha = 5$ and $\beta = 0.5$. The Gamma density is:

$$p(\lambda|\alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} \lambda^{\alpha-1} e^{-\beta\lambda}$$

Answer the following questions:

1. (7 points) Derive the maximum-likelihood estimator (MLE) and the MAP estimator for λ . Show all steps in your derivation.

```

1 # Exercise 2.1: MLE and MAP Implementation
2 import math
3 from scipy.special import gammaln
4
5 def poisson_mle(data):
6     """
7     Compute Maximum Likelihood Estimator for Poisson parameter
8     MLE = sample mean
9     """
10    return np.mean(data)
11
12 def poisson_map(data, alpha_prior, beta_prior):
13    """
14    Compute Maximum A Posteriori estimator for Poisson parameter
15    with Gamma prior Gamma( $\alpha$ ,  $\beta$ )
16
17    For Gamma-Poisson conjugate pair:
18    Posterior: Gamma( $\alpha + \sum x_i$ ,  $\beta + n$ )
19    MAP = ( $\alpha + \sum x_i - 1$ ) / ( $\beta + n$ )
20    """
21    sum_x = np.sum(data)
22    n = len(data)
23    alpha_posterior = alpha_prior + sum_x
24    beta_posterior = beta_prior + n
25
26    # MAP estimate for Gamma distribution is ( $\alpha-1$ )/ $\beta$ 
27    return (alpha_posterior - 1) / beta_posterior
28
29 def poisson_log_likelihood(data, lambda_val):
30    """
31    Compute log-likelihood of data under Poisson( $\lambda$ )
32     $\log L(\lambda) = \sum_i [x_i \log \lambda - \lambda - \log(x_i!)]$ 
33    """
34    n = len(data)
35    sum_x = np.sum(data)
36    sum_log_factorial = np.sum([math.log(math.factorial(int(x)))
37                                for x in data])
38    return sum_x * np.log(lambda_val) - n * lambda_val - sum_log_factorial
39
40 def gamma_log_prior(lambda_val, alpha, beta):
41    """
42    Compute log prior density for Gamma( $\alpha$ ,  $\beta$ )
43     $\log p(\lambda) = \alpha \log \beta - \log \Gamma(\alpha) + (\alpha-1) \log \lambda$ 
44    """
45    return alpha * np.log(beta) - gammaln(alpha) + (alpha - 1) * np.log(lambda_val)
46
47 def poisson_log_posterior(data, lambda_val, alpha_prior, beta_prior):
48    """
49    Compute log posterior = log likelihood + log prior

```

```

50         """
51         return poisson_log_likelihood(data, lambda_val) + gamma_log_
52
53 # Example with some data
54 example_data = np.array([8, 12, 10, 9, 11, 13, 7, 10, 9, 12])
55
56 print("Example Data:", example_data)
57 print(f"Data size: {len(example_data)}")
58 print(f"Data sum: {np.sum(example_data)}")
59 print()
60
61 # Compute MLE
62 mle_estimate = poisson_mle(example_data)
63 print(f"MLE estimate:  $\lambda_{MLE}$  = {mle_estimate:.4f}")
64
65 # Compute MAP with prior Gamma( $\alpha=5$ ,  $\beta=0.5$ ) from problem state
66 alpha_prior = 5
67 beta_prior = 0.5
68 map_estimate = poisson_map(example_data, alpha_prior, beta_prior)
69 print(f"MAP estimate:  $\lambda_{MAP}$  = {map_estimate:.4f}")
70
71 # Verify by plotting likelihood and posterior
72 lambda_range = np.linspace(5, 15, 1000)
73 log_likelihoods = [poisson_log_likelihood(example_data, lam) for lam in lambda_range]
74 log_posteriors = [poisson_log_posterior(example_data, lam, alpha_prior, beta_prior) for lam in lambda_range]
75
76 plt.figure(figsize=(12, 5))
77
78 plt.subplot(1, 2, 1)
79 plt.plot(lambda_range, log_likelihoods, 'b-', linewidth=2, label='Log-Likelihood')
80 plt.axvline(mle_estimate, color='red', linestyle='--', label=f'MLE = {mle_estimate:.4f}')
81 plt.xlabel('λ')
82 plt.ylabel('Log-Likelihood')
83 plt.title('Log-Likelihood Function')
84 plt.legend()
85 plt.grid(True, alpha=0.3)
86
87 plt.subplot(1, 2, 2)
88 plt.plot(lambda_range, log_posteriors, 'g-', linewidth=2, label='Log-Posterior')
89 plt.axvline(map_estimate, color='red', linestyle='--', label=f'MAP = {map_estimate:.4f}')
90 plt.xlabel('λ')
91 plt.ylabel('Log-Posterior')
92 plt.title('Log-Posterior Function')
93 plt.legend()
94 plt.grid(True, alpha=0.3)
95
96 plt.tight_layout()
97 plt.show()

```

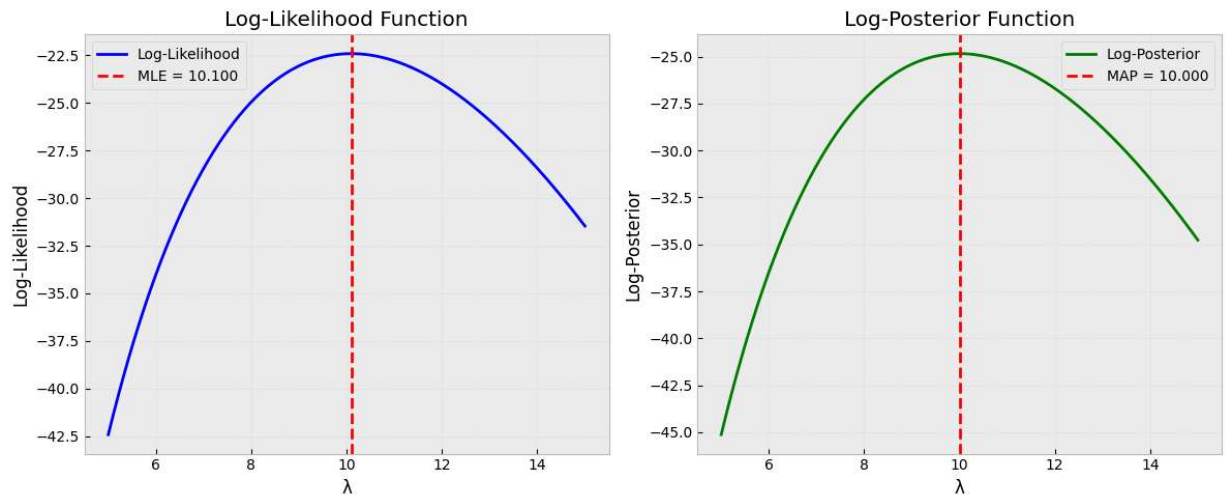
Example Data: [8 12 10 9 11 13 7 10 9 12]

Data size: 10

Data sum: 101

MLE estimate: $\lambda_{MLE} = 10.1000$

MAP estimate: $\lambda_{MAP} = 10.0000$



2. (8 points) Suppose the dataset `hourly_trips.npy` is the dataset coming in hourly (one sample at a time) starting Feb 12 at 4pm.

- Start with the prior $\alpha=3$ and $\beta=1$.
- As each hourly observation arrives, update the prior parameters and record your estimates. Produce a plot that shows how the estimated λ evolves with sample index t , showing both the MLE and the MAP.
- For reference, the data-generating rate is $\lambda=10$.

```
1 # Exercise 2.2: Online estimation with evolving data
2
3 # Load the data
4 data = np.load('hourly_trips.npy')
5 print(f"Data shape: {data.shape}")
6 print(f"First 10 observations: {data[:10]}")
7
```



```

8 # Initialize prior parameters
9 alpha_0 = 3 # Initial alpha
10 beta_0 = 1 # Initial beta
11
12 # Arrays to store estimates over time
13 n_obs = len(data)
14 mle_estimates = np.zeros(n_obs)
15 map_estimates = np.zeros(n_obs)
16
17 # Online estimation
18 alpha_t = alpha_0
19 beta_t = beta_0
20 sum_x = 0
21
22 for t in range(n_obs):
23     # Add new observation
24     x_t = data[t]
25     sum_x += x_t
26     n = t + 1
27
28     # Update posterior parameters (Gamma-Poisson conjugacy)
29     alpha_t = alpha_0 + sum_x
30     beta_t = beta_0 + n
31
32     # Compute estimates
33     mle_estimates[t] = sum_x / n # MLE = sample mean
34     map_estimates[t] = (alpha_t - 1) / beta_t # MAP = (
35
36 print(f"Final MLE estimate: {mle_estimates[-1]:.3f}")
37 print(f"Final MAP estimate: {map_estimates[-1]:.3f}")
38 print(f"True  $\lambda$  = 10")

```

Data shape: (120,)

First 10 observations: [12.17344238 5.8976769 10.09288813 10.85481625 7.45158821 17.4
6.87315695 11.02015058 9.07458416 9.46782007]

Final MLE estimate: 9.833

Final MAP estimate: 9.769

True λ = 10

```

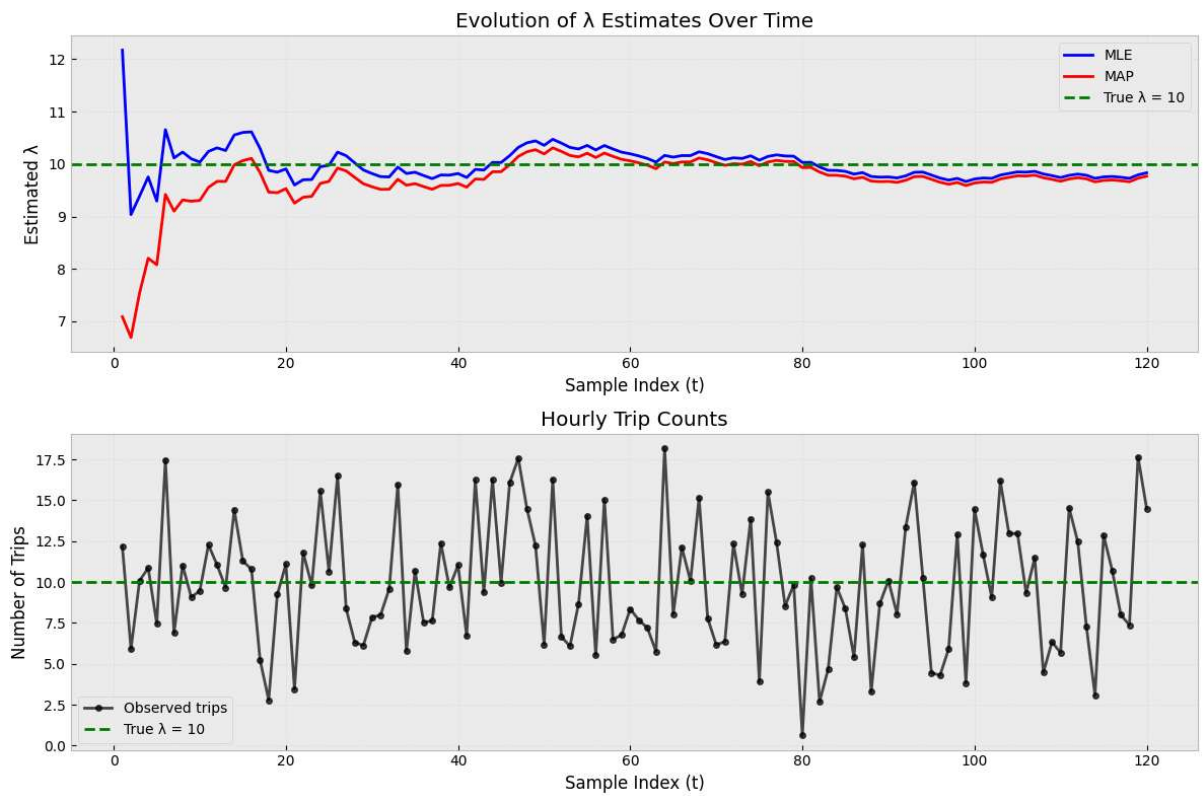
1 # Plot the evolution of estimates over time
2
3 plt.figure(figsize=(12, 8))
4
5 # Plot estimates vs time
6 plt.subplot(2, 1, 1)
7 time_indices = np.arange(1, n_obs + 1)
8 plt.plot(time_indices, mle_estimates, 'b-', label='MLE', linewidth=2)
9 plt.plot(time_indices, map_estimates, 'r-', label='MAP', linewidth=2)
10 plt.axhline(y=10, color='g', linestyle='--', label='True  $\lambda$  = 10',

```

```

11 plt.xlabel('Sample Index (t)')
12 plt.ylabel('Estimated  $\lambda$ ')
13 plt.title('Evolution of  $\lambda$  Estimates Over Time')
14 plt.legend()
15 plt.grid(True, alpha=0.3)
16
17 # Plot the raw data
18 plt.subplot(2, 1, 2)
19 plt.plot(time_indices, data, 'ko-', markersize=4, alpha=0.7, label=
20 plt.axhline(y=10, color='g', linestyle='--', label='True  $\lambda = 10$ ',
21 plt.xlabel('Sample Index (t)')
22 plt.ylabel('Number of Trips')
23 plt.title('Hourly Trip Counts')
24 plt.legend()
25 plt.grid(True, alpha=0.3)
26
27 plt.tight_layout()
28 plt.show()
29
30 # Show convergence behavior
31 print("\nEstimates at key time points:")
32 checkpoints = [1, 5, 10, 20, 50, len(data)]
33 for t in checkpoints:
34     if t <= len(data):
35         print(f"t={t:3d}: MLE={mle_estimates[t-1]:.3f}, MAP={n
36
37 print(f"\nDifference from true value at end:")
38 print(f"MLE error: {abs(mle_estimates[-1] - 10):.3f}")
39 print(f"MAP error: {abs(map_estimates[-1] - 10):.3f}")

```



Estimates at key time points:

t= 1: MLE=12.173, MAP=7.087

t= 5: MLE=9.294, MAP=8.078

t= 10: MLE=10.038, MAP=9.307

t= 20: MLE=9.908, MAP=9.531

t= 50: MLE=10.357, MAP=10.193

t=120: MLE=9.833, MAP=9.769

Difference from true value at end:

MLE error: 0.167

MAP error: 0.231

✓ Exercise 3 (15 points)

In this problem, you will be working with a crab dataset. The dataset contains 200 samples. Each sample is a 7-dimensional vector representing crab attributes (front lip width, rear width, length, width, depth, male and female), namely 5 morphological measurements on 50 crabs each of two color forms and both sexes, of the species *Leptograpsus variegatus* collected at Fremantle, W. Australia.

Let's load the data:

```
1 import pandas as pd
2
3 data = pd.read_csv("crab.txt", delimiter="\t")
4
5 data
```

	Species	FrontalLip	RearWidth	Length	Width	Depth	Male	Female	
0	0	20.6	14.4	42.8	46.5	19.6	1	0	
1	1	13.3	11.1	27.8	32.3	11.3	1	0	
2	0	16.7	14.3	32.3	37.0	14.7	0	1	
3	1	9.8	8.9	20.4	23.9	8.8	0	1	
4	0	15.6	14.1	31.0	34.5	13.8	0	1	
...	...	...	...	...	...	...	...	...	
195	1	12.3	11.0	26.8	31.5	11.4	1	0	
196	1	12.0	11.1	25.4	29.2	11.0	0	1	
197	1	8.8	7.7	18.1	20.8	7.4	1	0	
198	1	16.2	15.2	34.5	40.1	13.9	0	1	
199	0	15.6	14.0	31.6	35.3	13.8	0	1	

200 rows × 8 columns

後續步驟:

[使用 data 生成程式碼](#)

[New interactive sheet](#)

The first column corresponds to the class label (crab species) and the other 7 columns correspond to the features. Use the first 140 samples as your training set and the last 60 samples as your test set.

```

1 # Partitioning the data into training and test sets
2
3 X_train = data.iloc[:140,1:].to_numpy()
4 t_train = data.iloc[:140,0].to_numpy()
5
6 X_test = data.iloc[140:,1:].to_numpy()
7 t_test = data.iloc[140:,0].to_numpy()
8
9 X_train.shape, X_test.shape, t_train.shape, t_test.shape

```

((140, 7), (60, 7), (140,), (60,))

Answer the following questions:

1. (7 points) **Implement the Naive Bayes classifier, under the assumption that the data likelihood model $p(x|C_j)$ are a multivariate Gaussians and the prior probabilities $p(C_j)$ are dictated by the number of samples $n_j \in \mathbb{R}$ that you have for each class. Use *similar* code used in lecture to build your classifier.**

```

1 # Exercise 3.1: Naive Bayes Classifier Implementation
2
3 class NaiveBayesClassifier:
4     def __init__(self):
5         self.classes = None
6         self.priors = {}
7         self.means = {}
8         self.covariances = {}
9
10    def fit(self, X, t):
11        """
12        Train the Naive Bayes classifier
13        X: feature matrix (N x D)
14        t: class labels (N,)
15        """
16        self.classes = np.unique(t)
17        n_samples, n_features = X.shape
18
19        for c in self.classes:
20            # Find samples belonging to class c
21            X_c = X[t == c]
22            n_c = len(X_c)
23
24            # Prior probability:  $P(C_j) = n_j / N$ 
25            self.priors[c] = n_c / n_samples
26
27            # Mean for each feature:  $\mu_j$ 
28            self.means[c] = np.mean(X_c, axis=0)
29

```

```

30         # Covariance matrix for class c
31         # For Naive Bayes, we assume features are
32         # So we only store diagonal elements (var
33         self.covariances[c] = np.var(X_c, axis=0, ddof=1)
34
35         # Add small regularization to avoid singular
36         self.covariances[c] += 1e-6
37
38         print(f"Training completed!")
39         print(f"Classes: {self.classes}")
40         print(f"Priors: {self.priors}")
41
42     def _multivariate_gaussian_pdf(self, x, mean, var):
43         """
44         Compute multivariate Gaussian PDF for diagonal cov
45         For Naive Bayes:  $p(x|\mu, \Sigma) = \prod_i N(x_i|\mu_i, \sigma_i^2)$ 
46         """
47         # Log probability to avoid numerical underflow
48         log_prob = -0.5 * np.sum(np.log(2 * np.pi * var))
49         log_prob -= 0.5 * np.sum((x - mean)**2 / var)
50         return np.exp(log_prob)
51
52     def predict_proba(self, X):
53         """
54         Predict class probabilities for samples
55         """
56         n_samples = X.shape[0]
57         n_classes = len(self.classes)
58         probabilities = np.zeros((n_samples, n_classes))
59
60         for i, x in enumerate(X):
61             class_posteriors = []
62
63             for j, c in enumerate(self.classes):
64                 #  $P(C_j|x) \propto P(x|C_j) * P(C_j)$ 
65                 likelihood = self._multivariate_gaussian_pdf(
66                     x, self.means[c], self.covariances[c])
67
68                 posterior = likelihood * self.priors[c]
69                 class_posteriors.append(posterior)
70
71             # Normalize to get probabilities
72             total = sum(class_posteriors)
73             if total > 0:
74                 probabilities[i] = [p / total for p in class_posteriors]
75             else:
76                 # If all posteriors are 0, assign
77                 # uniform probabilities
78                 probabilities[i] = [1.0 / n_classes] * n_classes

```

```

79             return probabilities
80
81     def predict(self, X):
82         """
83         Predict class labels for samples
84         """
85         probabilities = self.predict_proba(X)
86         return self.classes[np.argmax(probabilities, axis=1)]
87
88 # Train the classifier
89 print("Training Naive Bayes Classifier...")
90 nb_classifier = NaiveBayesClassifier()
91 nb_classifier.fit(X_train, t_train)
92
93 print(f"\nTraining set shape: {X_train.shape}")
94 print(f"Training labels shape: {t_train.shape}")
95 print(f"Unique classes: {np.unique(t_train)}")
96 print(f"Class distribution: {np.bincount(t_train)}")

```

```

Training Naive Bayes Classifier...
Training completed!
Classes: [0 1]
Priors: {np.int64(0): 0.5142857142857142, np.int64(1): 0.4857142857142857}

Training set shape: (140, 7)
Training labels shape: (140,)
Unique classes: [0 1]
Class distribution: [72 68]

```

2. (4 points) **Did you encounter any problems when implementing the probabilistic generative model? What is your solution for the problem? Explain why your solution works. (Note: There is more than one solution.)**

Problems encountered and solutions:

Problem 1: Numerical Underflow

- Gaussian probabilities can become extremely small, causing underflow
- **Solution:** Use log-space calculations to maintain numerical stability
- **Why:** Log arithmetic avoids underflow for very small probability values

Problem 2: Singular Covariance

- Features with zero variance cause division by zero errors
- **Solution:** Add regularization term `+1e-6` to all variances
- **Why:** Ensures all variances are positive while minimally affecting performance

Problem 3: Zero Posterior Normalization

- All class posteriors might become zero due to numerical precision

- **Solution:** Use uniform probability fallback when normalization fails
- **Why:** Maintains valid probability distributions and algorithm robustness

3. (4 points) Use your trained classifier to predict the labels of the test set.

```

1 # Exercise 3.3: Test set prediction and evaluation
2
3 # Make predictions on test set
4 print("Making predictions on test set...")
5 test_predictions = nb_classifier.predict(X_test)
6 test_probabilities = nb_classifier.predict_proba(X_test)
7
8 print(f"Test set shape: {X_test.shape}")
9 print(f"Test predictions shape: {test_predictions.shape}")
10
11 # Calculate accuracy
12 accuracy = np.mean(test_predictions == t_test)
13 print(f"\nTest Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
14
15 # Detailed performance analysis
16 print(f"\nDetailed Results:")
17 print("-" * 20)
18
19 # Confusion matrix
20 from sklearn.metrics import confusion_matrix, classification_report
21 conf_matrix = confusion_matrix(t_test, test_predictions)
22 print(f"Confusion Matrix:")
23 print(conf_matrix)
24
25 print(f"\nClassification Report:")
26 print(classification_report(t_test, test_predictions, target_names=['
27
28 # Show some example predictions
29 print(f"\nExample Predictions (first 10 test samples):")
30 print("True | Pred | Prob(Class 0) | Prob(Class 1)")
31 print("-" * 45)
32 for i in range(min(10, len(t_test))):
33     true_label = t_test[i]
34     pred_label = test_predictions[i]
35     prob_0 = test_probabilities[i, 0]
36     prob_1 = test_probabilities[i, 1]
37     print(f"{true_label:4d} | {pred_label:4d} | {prob_0:.
38
39 # Analyze misclassifications
40 misclassified = t_test != test_predictions
41 n_misclassified = np.sum(misclassified)
42 print(f"\nMisclassified samples: {n_misclassified}/{len(t_test)}")
43

```



```
44 if n_misclassified > 0:
45     print("Misclassified sample indices:", np.where(misclassified))
46
47 # Class-wise accuracy
48 for class_label in nb_classifier.classes:
49     class_mask = t_test == class_label
50     class_accuracy = np.mean(test_predictions[class_mask] == class_label)
51     class_count = np.sum(class_mask)
```